

Student Lab Manual

Azure VM, Kafka & .NET Microservices

Real-Time Payment Feedback with Bicep, Kafka KRaft and .NET 8

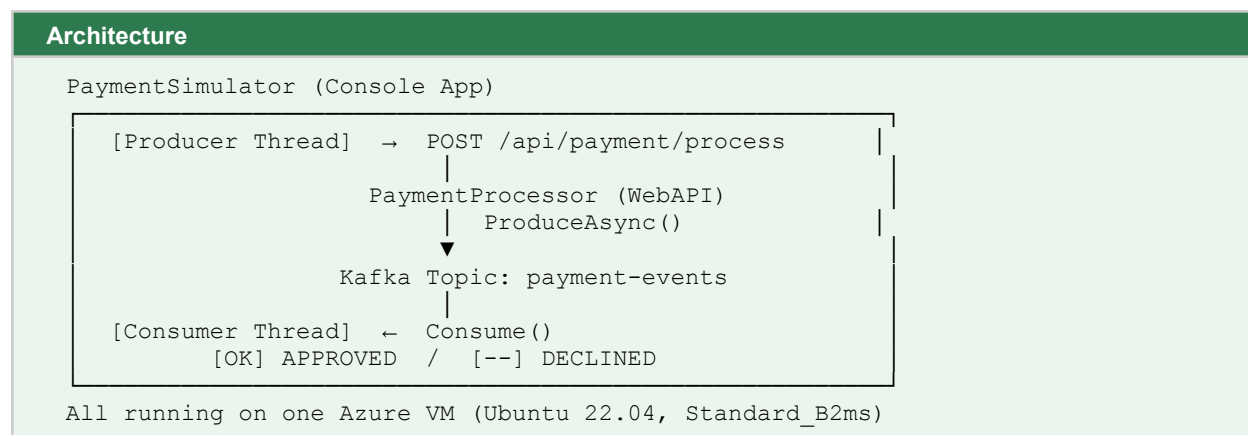
Estimated Duration: 2.5 hours

Lab Overview

In this lab you will provision a Linux VM on Azure using a Bicep template — an Infrastructure as Code approach that eliminates manual installation steps. Once the VM is ready, you will build two .NET 8 applications directly inside the VM:

- **PaymentProcessor** — an ASP.NET Core WebAPI that receives a payment request, randomly approves or declines it (like a mock payment gateway), and publishes the result to a Kafka topic.
- **PaymentSimulator** — a console app that continuously sends random payment requests to the WebAPI, and simultaneously reads results from Kafka, displaying them in real time with colour-coded output.

Architecture



What You Will Learn

- How to write and deploy a Bicep template with a Custom Script Extension
- How Kafka works in KRaft mode (no ZooKeeper)
- How to produce and consume messages using Confluent.Kafka for .NET
- How the producer / consumer pattern enables real-time event streaming

Prerequisites Checklist

Confirm each item before starting:

- ☐ Azure subscription with Contributor role assigned
- ☐ Azure CLI installed and signed in — run `az account show` to confirm
- ☐ Windows Command Prompt (`cmd.exe`) or Windows Terminal
- ☐ SSH client available — test by typing `ssh` in `cmd.exe` (should show usage, not an error)
- ☐ A text editor — Notepad, VS Code, or any editor that can save plain text files

Part 1: Provision Infrastructure with Bicep

INFO

Bicep is Azure's Infrastructure as Code language. Instead of clicking through the portal, you describe your resources in a .bicep file and Azure creates them.

In this lab the Bicep template also attaches a Custom Script Extension — a mechanism that runs a bash script on the VM automatically after it is created. The script installs Java 17, .NET 8 SDK, and Apache Kafka 3.7. By the time deployment finishes, everything is installed and Kafka is already running as a background service.

Step 1.1 — Set Variables

Open a Command Prompt window. Set these variables — they are used in all subsequent commands.

CMD (Windows Command Prompt)

```
set RG=kafka-lab-rg
set LOCATION=eastus
set VM_NAME=kafka-lab-vm
set ADMIN_USER=azureuser
set ADMIN_PASS=KafkaLab@2024!
```

RECORD YOUR VALUES

RG =	<i>abubakkar-kafka-lab-rg</i>
LOCATION =	<i>eastus</i>
ADMIN_PASS =	<i>KafkaLab@2024</i>

Step 1.2 — Create Resource Group

CMD (Windows Command Prompt)

```
az group create --name %RG% --location %LOCATION%
```

Step 1.3 — Create Project Folder

CMD (Windows Command Prompt)

```
mkdir %USERPROFILE%\kafka-lab
cd %USERPROFILE%\kafka-lab
```

Step 1.4 — Create main.bicep

Open a text editor and save the following as main.bicep inside the kafka-lab folder:

BICEP (main.bicep)

```
param location string = 'eastus'
param vmName string = 'kafka-lab-vm'
param adminUsername string = 'azureuser'

@secure()
param adminPassword string

param vmSize string = 'Standard_B2ms'

var nsgName = '${vmName}-nsg'
var vnetName = '${vmName}-vnet'
var subnetName = 'default'
var publicIpName = '${vmName}-pip'
var nicName = '${vmName}-nic'

resource nsg 'Microsoft.Network/networkSecurityGroups@2023-04-01' = {
  name: nsgName
  location: location
  properties: {
    securityRules: [{
      name: 'AllowSSH'
      properties: {
        priority: 1000
        protocol: 'Tcp'
        access: 'Allow'
        direction: 'Inbound'
        sourceAddressPrefix: '*'
        sourcePortRange: '*'
        destinationAddressPrefix: '*'
        destinationPortRange: '22'
      }
    }]
  }
}

resource vnet 'Microsoft.Network/virtualNetworks@2023-04-01' = {
  name: vnetName
  location: location
  properties: {
    addressSpace: { addressPrefixes: ['10.0.0.0/16'] }
    subnets: [{
      name: subnetName
      properties: { addressPrefix: '10.0.0.0/24', networkSecurityGroup: { id:
nsg.id } }
    }]
  }
}

resource publicIp 'Microsoft.Network/publicIPAddresses@2023-04-01' = {
```

```
name: publicIpName
location: location
sku: { name: 'Standard' }
properties: { publicIPAllocationMethod: 'Static' }
}

resource nic 'Microsoft.Network/networkInterfaces@2023-04-01' = {
  name: nicName
  location: location
  properties: {
    ipConfigurations: [{
      name: 'ipconfig1'
      properties: {
        subnet: { id: '${vnet.id}/subnets/${subnetName}' }
        publicIPAddress: { id: publicIp.id }
      }
    }]
  }
}

resource vm 'Microsoft.Compute/virtualMachines@2023-07-01' = {
  name: vmName
  location: location
  properties: {
    hardwareProfile: { vmSize: vmSize }
    osProfile: {
      computerName: vmName
      adminUsername: adminUsername
      adminPassword: adminPassword
    }
    storageProfile: {
      imageReference: {
        publisher: 'Canonical'
        offer: '0001-com-ubuntu-server-jammy'
        sku: '22_04-lts-gen2'
        version: 'latest'
      }
      osDisk: {
        createOption: 'FromImage'
        managedDisk: { storageAccountType: 'StandardSSD_LRS' }
      }
    }
    networkProfile: { networkInterfaces: [{ id: nic.id }] }
  }
}

resource setup 'Microsoft.Compute/virtualMachines/extensions@2023-07-01' = {
  parent: vm
  name: 'setup-kafka'
  location: location
  properties: {
    publisher: 'Microsoft.Azure.Extensions'
    type: 'CustomScript'
    typeHandlerVersion: '2.1'
    autoUpgradeMinorVersion: true
    settings: {}
    protectedSettings: {
      script: base64(loadTextContent('setup.sh'))
    }
  }
}
```

```
output publicIpAddress string = publicIp.properties.ipAddress
output sshCommand      string = 'ssh
${adminUsername}@${publicIp.properties.ipAddress}'
```

Step 1.5 — Create setup.sh

In the same kafka-lab folder, create a file called setup.sh with the following content. This is the script that installs everything on the VM automatically.

BASH SCRIPT (setup.sh)

```
#!/bin/bash
set -e
exec > /var/log/kafka-setup.log 2>&1

echo "=== Kafka Lab Setup Started ==="
date

# 1. System packages
apt-get update -y
apt-get install -y openjdk-17-jdk wget curl

# 2. Set JAVA_HOME system-wide
echo "JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64" >> /etc/environment
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64

# 3. Install .NET 8 SDK
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb \
    -O /tmp/ms-prod.deb
dpkg -i /tmp/ms-prod.deb
apt-get update -y
apt-get install -y dotnet-sdk-8.0

# 4. Download and extract Kafka 3.7.0
cd /opt
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz -O kafka.tgz
tar -xzf kafka.tgz
mv kafka_2.13-3.7.0 kafka
rm kafka.tgz

# 5. Format KRaft storage (single-node, no ZooKeeper)
KAFKA_UUID=$(JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-storage.sh random-uuid)
JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-storage.sh format \
    -t "$KAFKA_UUID" \
    -c /opt/kafka/config/kraft/server.properties

# 6. Create systemd service
cat > /etc/systemd/system/kafka.service << EOF
[Unit]
Description=Apache Kafka (KRaft Mode)
After=network.target

[Service]
Type=simple
User=root
Environment=JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

```

ExecStart=/opt/kafka/bin/kafka-server-start.sh
/opt/kafka/config/kraft/server.properties
ExecStop=/opt/kafka/bin/kafka-server-stop.sh
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable kafka
systemctl start kafka

# 7. Wait for Kafka then create the topic
MAX_WAIT=90; WAITED=0
until JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-topics.sh \
  --list --bootstrap-server localhost:9092 > /dev/null 2>&1; do
  [ $WAITED -ge $MAX_WAIT ] && echo "ERROR: Kafka timeout" && exit 1
  echo "Waiting for Kafka... (${WAITED}s elapsed)"
  sleep 5; WAITED=$((WAITED+5))
done

JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-topics.sh \
  --create --topic payment-events \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1

echo "=== Setup Complete ===" && date

```

Step 1.6 — Deploy the Bicep Template

Make sure you are in the kafka-lab folder, then run:

CMD (Windows Command Prompt)

```

cd %USERPROFILE%\kafka-lab

az deployment group create ^
  --resource-group %RG% ^
  --name kafka-deploy ^
  --template-file main.bicep ^
  --parameters adminPassword="%ADMIN_PASS%"

```

INFO

This deployment takes approximately 10-14 minutes. Azure is:

1. Creating the VM, VNet, NSG, NIC and public IP (~2-3 min)
2. Running setup.sh via the Custom Script Extension (~8-10 min)
 setup.sh installs Java, .NET SDK, and Kafka — then starts the Kafka service.

You will see "Running..." in the terminal while it works. When it finishes,

the output section shows publicIpAddress and sshCommand.

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars Kafka-Lab % az group create --name $RG --location $LOCATION
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abubakkar-kafka-lab-rg",
  "location": "eastus2",
  "managedBy": null,
  "name": "abubakkar-kafka-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars Kafka-Lab % az deployment group create \
--resource-group $RG \
--name kafka-deploy \
--template-file main.bicep \
--parameters adminPassword="$ADMIN_PASS"
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abubakkar-kafka-lab-rg/providers/Microsoft.Resources/deployments/kafka-deploy",
  "location": null,
  "name": "kafka-deploy",
  "properties": {
    "correlationId": "932ffe10-67e5-4179-bcfd-5533dfc2e649",
    "debugSetting": null,
    "dependencies": [
      {
        "dependsOn": [
          {
            "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abubakkar-kafka-lab-rg/providers/Microsoft.Network/networkSecurityGroups/abubakkar-kafka-vm-nsg",
            "resourceGroup": "abubakkar-kafka-lab-rg",
            "resourceName": "abubakkar-kafka-vm-nsg",
            "resourceType": "Microsoft.Network/networkSecurityGroups"
          }
        ]
      }
    ]
  }
}
```

Step 1.7 — Capture the VM IP Address

When deployment finishes, capture the public IP from the outputs:

CMD (Windows Command Prompt)

```
for /f "delims=" %a in ('az deployment group show ^
--resource-group %RG% --name kafka-deploy ^
--query "properties.outputs.publicIpAddress.value" -o tsv') do set VM_IP=%a

echo VM IP: %VM_IP%
```

RECORD YOUR VALUES

VM_IP =	20.96.30.226
---------	--------------

Step 1.8 — Connect to the VM via SSH

CMD (Windows Command Prompt)

```
ssh %ADMIN_USER%%VM_IP%
```

(Type yes to accept the host key fingerprint)
(Enter password: KafkaLab@2024!)

INFO

You are now INSIDE the Linux VM. Your prompt changes to:

```
azureuser@kafka-lab-vm:~$
```

Everything from here until Part 4 runs in this SSH session.

Linux commands are case-sensitive — type them exactly as shown.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

```
abubakkar@abubakkar-kafka-vm:~$
```

```
abubakkar@abubakkar-kafka-vm:~$
```

Step 1.9 — Verify the Stack

Confirm Java, .NET, and Kafka are all ready:

BASH (SSH — Linux VM)

```
# Check Kafka service status
systemctl status kafka --no-pager
```

```
# Check Java version
java -version
```

```
# Check .NET version
dotnet --version
```

```
# List Kafka topics — should show: payment-events
/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

RECORD YOUR VALUES

Java version:	17.0.19
.NET version:	8.0.422
Kafka topics listed:	should be: payment-events

```

abubakkar@abubakkar-kafka-vm:~$
abubakkar@abubakkar-kafka-vm:~$ java --version
openjdk 17.0.19 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-22.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-22.04.2-Ubuntu, mixed mode, sharing)
abubakkar@abubakkar-kafka-vm:~$ dotnet --version
8.0.422
abubakkar@abubakkar-kafka-vm:~$ sudo systemctl status kafka
● kafka.service - Apache Kafka (KRaft Mode)
   Loaded: loaded (/etc/systemd/system/kafka.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-07-03 09:02:36 UTC; 6min ago
     Main PID: 6239 (java)
       Tasks: 97 (limit: 9522)
      Memory: 350.8M
         CPU: 8.566s
    CGroup: /system.slice/kafka.service
            └─6239 /usr/lib/jvm/java-17-openjdk-amd64/bin/java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -XX:InitiatingHeapOccupancyPercent=...

Jul 03 09:02:39 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:39,944] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka
Jul 03 09:02:39 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:39,944] INFO Kafka version: 3.7.0 (org.apache.kafka.common.utils.AppInfoPa
Jul 03 09:02:39 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:39,944] INFO Kafka commitId: 2ae524ed625438c5 (org.apache.kafka.common.ut
Jul 03 09:02:39 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:39,944] INFO Kafka startTimeMs: 1783069359944 (org.apache.kafka.common.ut
Jul 03 09:02:39 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:39,946] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server
Jul 03 09:02:41 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:41,940] INFO [ReplicaFetcherManager on broker 1] Removed fetcher for parti
Jul 03 09:02:41 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:41,968] INFO [LogLoader partition=payment-events-0, dir=/tmp/kraft-comb
Jul 03 09:02:41 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:41,973] INFO Created log for partition payment-events-0 in /tmp/kraft-comb
Jul 03 09:02:41 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:41,975] INFO [Partition payment-events-0 broker=1] No checkpointed highwat
Jul 03 09:02:41 abubakkar-kafka-vm kafka-server-start.sh[6239]: [2026-07-03 09:02:41,977] INFO [Partition payment-events-0 broker=1] Log loaded for partiti
lines 1-20/20 (END)

```

REFLECTION QUESTIONS

1. What does `loadTextContent('setup.sh')` do in the Bicep template? When does it run?

`loadTextContent('setup.sh')` reads the contents of the `setup.sh` file during deployment. The script is Base64 encoded and passed to the Azure Custom Script Extension, which executes it automatically after the virtual machine is created.

2. What is the role of the Custom Script Extension? How does Azure know what to run?

The Custom Script Extension allows Azure to run custom scripts on a VM after provisioning. Azure executes the script specified in the extension configuration, which in this lab is the Base64-encoded `setup.sh` script.

3. Why does KRaft mode need `kafka-storage.sh` format to run before Kafka can start?

KRaft mode stores Kafka metadata internally instead of using ZooKeeper. Before Kafka starts for the first time, `kafka-storage.sh` format initializes the storage and creates the required metadata directory using a unique cluster ID. Without this step, Kafka cannot start because its metadata storage has not been initialized.

Part 2: Build the PaymentProcessor WebAPI

This WebAPI acts as a mock payment gateway. It receives a payment request, randomly approves (~70%) or declines (~30%) it, publishes the result to Kafka, and returns a JSON response.

Step 2.1 — Create the WebAPI Project

BASH (SSH — Linux VM)

```
cd ~
dotnet new webapi -n PaymentProcessor --no-https
cd PaymentProcessor
```

Step 2.2 — Add Confluent.Kafka Package

BASH (SSH — Linux VM)

```
dotnet add package Confluent.Kafka
```

Step 2.3 — Create the Models

BASH (SSH — Linux VM)

```
mkdir -p Models

cat > Models/PaymentRequest.cs << 'EOF'
namespace PaymentProcessor.Models;

public class PaymentRequest
{
    public string TransactionId { get; set; } = string.Empty;
    public string MerchantName { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public string CardLastFour { get; set; } = string.Empty;
}
EOF

cat > Models/PaymentResult.cs << 'EOF'
namespace PaymentProcessor.Models;

public class PaymentResult
{
    public string TransactionId { get; set; } = string.Empty;
    public string MerchantName { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public string CardLastFour { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;
}
```

```

    public string Reason          { get; set; } = string.Empty;
    public DateTimeOffset Timestamp { get; set; }
}
EOF

```

Step 2.4 — Create the Payment Controller

BASH (SSH — Linux VM)

```

mkdir -p Controllers

cat > Controllers/PaymentController.cs << 'EOF'
using Confluent.Kafka;
using Microsoft.AspNetCore.Mvc;
using PaymentProcessor.Models;
using System.Text.Json;

namespace PaymentProcessor.Controllers;

[ApiController]
[Route("api/[controller]")]
public class PaymentController : ControllerBase
{
    private readonly IProducer<string, string> _producer;

    private static readonly string[] DeclineReasons =
    {
        "Insufficient funds",
        "Card blocked by issuer",
        "Suspected fraud - unusual activity",
        "Daily transaction limit exceeded",
        "Card expired"
    };

    public PaymentController(IProducer<string, string> producer)
        => _producer = producer;

    [HttpPost("process")]
    public async Task<IActionResult> ProcessPayment([FromBody] PaymentRequest req)
    {
        var rng          = new Random();
        var approved = rng.Next(0, 10) > 2;    // ~70% approval rate

        var result = new PaymentResult
        {
            TransactionId = req.TransactionId,
            MerchantName  = req.MerchantName,
            Amount        = req.Amount,
            CardLastFour  = req.CardLastFour,
            Status        = approved ? "APPROVED" : "DECLINED",
            Reason        = approved ? "Payment authorised"
                          :
DeclineReasons[rng.Next(DeclineReasons.Length)],
            Timestamp     = DateTimeOffset.UtcNow
        };

        await _producer.ProduceAsync("payment-events",
            new Message<string, string>

```

```
        {
            Key    = result.TransactionId,
            Value = JsonSerializer.Serialize(result)
        });

        return Ok(result);
    }

    [HttpGet("health")]
    public IActionResult Health()
        => Ok(new { status = "OK", time = DateTimeOffset.UtcNow });
    }
EOF
```

Step 2.5 — Replace Program.cs

BASH (SSH — Linux VM)

```
cat > Program.cs << 'EOF'
using Confluent.Kafka;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

var producerConfig = new ProducerConfig { BootstrapServers = "localhost:9092" };
builder.Services.AddSingleton<IProducer<string, string>>(
    new ProducerBuilder<string, string>(producerConfig).Build());

var app = builder.Build();
app.MapControllers();
app.Run("http://localhost:5100");
EOF
```

Step 2.6 — Remove Default Template Files

BASH (SSH — Linux VM)

```
rm -f WeatherForecast.cs Controllers/WeatherForecastController.cs
```

Step 2.7 — Build and Run

BASH (SSH — Linux VM)

```
dotnet run
```

You should see:

Expected Output

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

```
[abubakkar@abubakkar-kafka-vm:~/PaymentProcessor$ dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/abubakkar/PaymentProcessor
```

INFO

Leave this terminal running. The WebAPI must stay active for Part 3.
You will open a NEW SSH session for the console app.

REFLECTION QUESTIONS

1. Why is `IProducer<string, string>` registered as a Singleton (not Scoped or Transient)?

A Kafka producer is thread-safe and designed to be reused. Registering it as a Singleton creates a single producer instance for the entire application, reducing connection overhead and improving performance.

2. What do the two type parameters of `IProducer<string, string>` represent?

The first string represents the **message key**, and the second **string** represents the **message value (payload)**. In this lab, the transaction ID is used as the key, and the payment result JSON is used as the value.

3. The controller writes to Kafka after determining the result. Why might a real system

Writing to Kafka before sending the HTTP response helps ensure that the event has been published successfully. This reduces the risk of losing important events and allows downstream services to process the event reliably.

Part 3: Build the PaymentSimulator Console App

The simulator has two concurrent tasks running in parallel. One task sends a payment request to the WebAPI every 3 seconds. The other task listens on the Kafka topic and prints each payment result as it arrives — in real time.

Step 3.1 — Open a Second SSH Session

Leave the WebAPI terminal open. Open a new Command Prompt window on your machine and SSH in again:

CMD (Windows Command Prompt)

```
ssh %ADMIN_USER%@%VM_IP%
```

(If %VM_IP% is no longer set, re-run the capture command from Step 1.7)

Step 3.2 — Create the Console Project

BASH (SSH — Linux VM)

```
cd ~  
dotnet new console -n PaymentSimulator  
cd PaymentSimulator
```

Step 3.3 — Add Confluent.Kafka Package

BASH (SSH — Linux VM)

```
dotnet add package Confluent.Kafka
```

Step 3.4 — Replace Program.cs

BASH (SSH — Linux VM)

```
cat > Program.cs << 'EOF'  
using Confluent.Kafka;  
using System.Net.Http.Json;  
using System.Text.Json;  
  
const string WebApiUrl = "http://localhost:5100";  
const string KafkaBoot = "localhost:9092";  
const string TopicName = "payment-events";
```

```

string[] Merchants = { "Amazon UK", "Tesco Online", "Netflix",
                        "Uber Eats", "Sainsburys", "ASOS", "Spotify" };
string[] Cards = { "4242", "1234", "5678", "9999", "0011" };

var cts = new CancellationTokenSource();
var httpClient = new HttpClient { BaseAddress = new Uri(WebApiUrl) };
var rng = new Random();
int counter = 0;

Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

// — Kafka Consumer (background)
var consumerTask = Task.Run(() =>
{
    var cfg = new ConsumerConfig
    {
        BootstrapServers = KafkaBoot,
        GroupId = "payment-simulator-group",
        AutoOffsetReset = AutoOffsetReset.Latest
    };
    using var consumer = new ConsumerBuilder<string, string>(cfg).Build();
    consumer.Subscribe(TopicName);

    Console.ForegroundColor = ConsoleColor.Cyan;
    Console.WriteLine($"[Consumer] Subscribed to '{TopicName}'. Waiting...\n");
    Console.ResetColor();

    while (!cts.Token.IsCancellationRequested)
    {
        try
        {
            var cr = consumer.Consume(TimeSpan.FromSeconds(1));
            if (cr is null) continue;

            var r = JsonSerializer.Deserialize<PaymentResult>(
                cr.Message.Value,
                new JsonSerializerOptions { PropertyNameCaseInsensitive = true });

            Console.ForegroundColor = r.Status == "APPROVED"
                ? ConsoleColor.Green : ConsoleColor.Red;

            Console.WriteLine(
                $" {(r.Status == "APPROVED" ? "[OK]" : "[--])} " +
                $"[{r.Timestamp:HH:mm:ss}] {r.TransactionId} " +
                $"{r.MerchantName,-16} £{r.Amount,7:F2} " +
                $"****{r.CardLastFour} {r.Status,-8} {r.Reason}");

            Console.ResetColor();
        }
        catch (ConsumeException ex)
        when (!cts.Token.IsCancellationRequested)
        {
            Console.Error.WriteLine($"[Consumer error] {ex.Error.Reason}");
        }
    }
    consumer.Close();
});

await Task.Delay(2000);

Console.ForegroundColor = ConsoleColor.Yellow;

```

```

Console.WriteLine("=====");
Console.WriteLine("  Payment Simulator  --  Real-Time Feed      ");
Console.WriteLine("=====\\n");
Console.ResetColor();
Console.WriteLine("Sending one payment every 3 seconds. Ctrl+C to stop.\\n");

// — HTTP Producer loop —————
while (!cts.Token.IsCancellationRequested)
{
    var req = new {
        transactionId = $"TXN{++counter:D6}",
        merchantName  = Merchants[rng.Next(Merchants.Length)],
        amount        = Math.Round((decimal)(rng.NextDouble() * 249 + 1), 2),
        cardLastFour  = Cards[rng.Next(Cards.Length)]
    };

    Console.ForegroundColor = ConsoleColor.DarkYellow;
    Console.WriteLine($"-> [{DateTime.UtcNow:HH:mm:ss}] {req.transactionId}  " +
        $"{req.merchantName,-16} £{req.amount:F2}  ****{req.cardLastFour}");
    Console.ResetColor();

    try { await httpClient.PostAsJsonAsync("/api/payment/process", req, cts.Token); }
    catch (Exception ex) when (!cts.Token.IsCancellationRequested)
        { Console.Error.WriteLine($"[HTTP error] {ex.Message}"); }

    await Task.Delay(3000).ContinueWith(_ => { });
}

await consumerTask;
Console.WriteLine("\\nSimulator stopped.");

record PaymentResult(string TransactionId, string MerchantName,
    decimal Amount, string CardLastFour, string Status,
    string Reason, DateTimeOffset Timestamp);
EOF

```

Step 3.5 — Run the Simulator

BASH (SSH — Linux VM)

```
dotnet run
```

You will see outbound arrows (->) as payments are sent, and [OK]/[--] lines as results arrive from Kafka:

Expected Output (approximate)

```

=====
  Payment Simulator  --  Real-Time Feed      -----
=====

[Consumer] Subscribed to payment-events. Waiting...

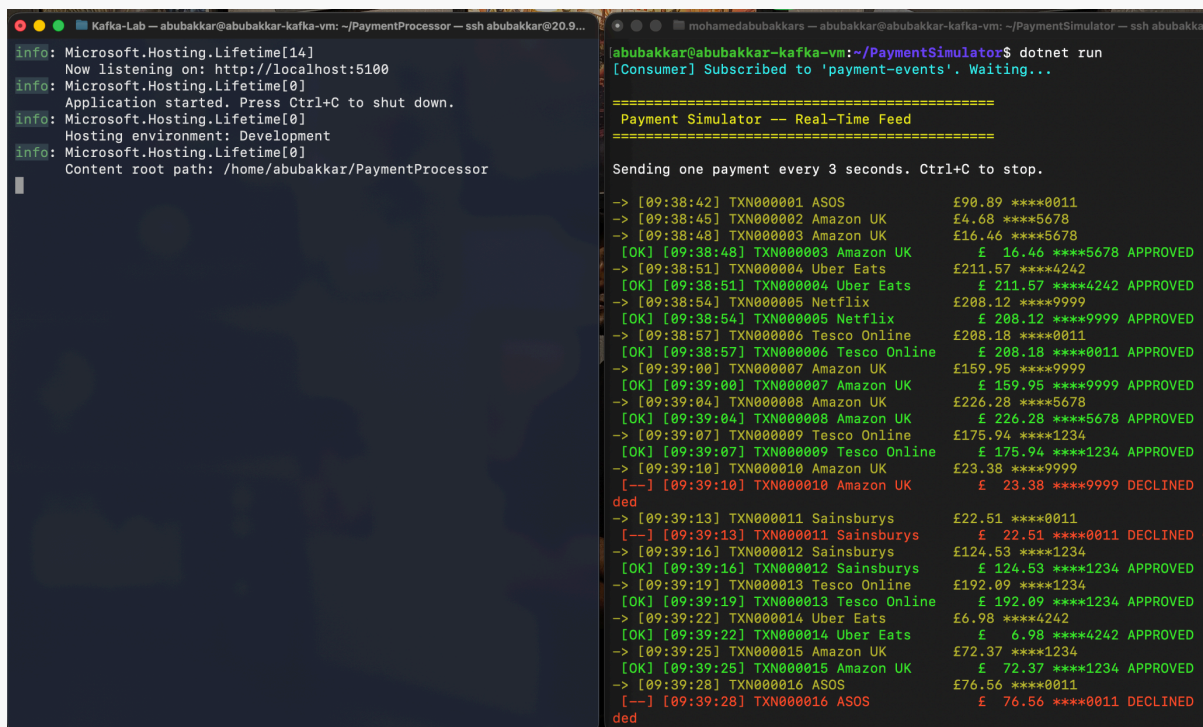
```

```

-> [14:23:01] TXN000001 Amazon UK          £127.45  ****4242
-> [14:23:04] TXN000002 Netflix            £89.99   ****1234

[OK] [14:23:01] TXN000001 Amazon UK          £127.45  ****4242 APPROVED
Payment authorised
[--] [14:23:04] TXN000002 Netflix            £89.99   ****1234 DECLINED
Insufficient funds

```



The screenshot shows two terminal windows. The left window displays the output of the PaymentProcessor application, which is listening on http://localhost:5100 and has started successfully. The right window displays the output of the PaymentSimulator application, which is running a real-time feed of payment events. The events include transactions for various retailers like ASOS, Amazon UK, Uber Eats, Netflix, and Tesco Online, with their respective amounts and approval statuses.

```

Kafka-Lab - abubakkar@abubakkar-kafka-vm: ~/PaymentProcessor - ssh abubakkar@20.9...
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: /home/abubakkar/PaymentProcessor

abubakkar@abubakkar-kafka-vm: ~/PaymentSimulator$ dotnet run
[Consumer] Subscribed to 'payment-events'. Waiting...

Payment Simulator -- Real-Time Feed

Sending one payment every 3 seconds. Ctrl+C to stop.

-> [09:38:42] TXN000001 ASOS          £90.89  ****0011
-> [09:38:45] TXN000002 Amazon UK     £4.68   ****5678
-> [09:38:48] TXN000003 Amazon UK     £16.46  ****5678
[OK] [09:38:48] TXN000003 Amazon UK     £ 16.46  ****5678 APPROVED
-> [09:38:51] TXN000004 Uber Eats     £211.57  ****4242
[OK] [09:38:51] TXN000004 Uber Eats     £ 211.57  ****4242 APPROVED
-> [09:38:54] TXN000005 Netflix       £208.12  ****9999
[OK] [09:38:54] TXN000005 Netflix       £ 208.12  ****9999 APPROVED
-> [09:38:57] TXN000006 Tesco Online  £208.18  ****0011
[OK] [09:38:57] TXN000006 Tesco Online  £ 208.18  ****0011 APPROVED
-> [09:39:00] TXN000007 Amazon UK     £159.95  ****9999
[OK] [09:39:00] TXN000007 Amazon UK     £ 159.95  ****9999 APPROVED
-> [09:39:04] TXN000008 Amazon UK     £226.28  ****5678
[OK] [09:39:04] TXN000008 Amazon UK     £ 226.28  ****5678 APPROVED
-> [09:39:07] TXN000009 Tesco Online  £175.94  ****1234
[OK] [09:39:07] TXN000009 Tesco Online  £ 175.94  ****1234 APPROVED
-> [09:39:10] TXN000010 Amazon UK     £23.38  ****9999
[--] [09:39:10] TXN000010 Amazon UK     £ 23.38  ****9999 DECLINED
ded
-> [09:39:13] TXN000011 Sainsburys    £22.51  ****0011
[--] [09:39:13] TXN000011 Sainsburys    £ 22.51  ****0011 DECLINED
-> [09:39:16] TXN000012 Sainsburys    £124.53  ****1234
[OK] [09:39:16] TXN000012 Sainsburys    £ 124.53  ****1234 APPROVED
-> [09:39:19] TXN000013 Tesco Online  £192.09  ****1234
[OK] [09:39:19] TXN000013 Tesco Online  £ 192.09  ****1234 APPROVED
-> [09:39:22] TXN000014 Uber Eats     £6.98    ****4242
[OK] [09:39:22] TXN000014 Uber Eats     £ 6.98    ****4242 APPROVED
-> [09:39:25] TXN000015 Amazon UK     £72.37  ****1234
[OK] [09:39:25] TXN000015 Amazon UK     £ 72.37  ****1234 APPROVED
-> [09:39:28] TXN000016 ASOS          £76.56  ****0011
[--] [09:39:28] TXN000016 ASOS          £ 76.56  ****0011 DECLINED
ded

```

```
abubakkar@abubakkar-kafka-vm:~/PaymentSimulator$ dotnet run
[Consumer] Subscribed to 'payment-events'. Waiting...

=====
Payment Simulator -- Real-Time Feed
=====

Sending one payment every 3 seconds. Ctrl+C to stop.

-> [09:38:42] TXN000001 ASOS                £90.89 *****0011
-> [09:38:45] TXN000002 Amazon UK            £4.68 *****5678
-> [09:38:48] TXN000003 Amazon UK            £16.46 *****5678
[OK] [09:38:48] TXN000003 Amazon UK          £ 16.46 *****5678 APPROVED Payment authorised
-> [09:38:51] TXN000004 Uber Eats            £211.57 ****4242
[OK] [09:38:51] TXN000004 Uber Eats          £ 211.57 *****4242 APPROVED Payment authorised
-> [09:38:54] TXN000005 Netflix              £208.12 ****9999
[OK] [09:38:54] TXN000005 Netflix            £ 208.12 *****9999 APPROVED Payment authorised
-> [09:38:57] TXN000006 Tesco Online          £208.18 *****011
[OK] [09:38:57] TXN000006 Tesco Online        £ 208.18 *****011 APPROVED Payment authorised
-> [09:39:00] TXN000007 Amazon UK            £159.95 ****9999
[OK] [09:39:00] TXN000007 Amazon UK          £ 159.95 ****9999 APPROVED Payment authorised
-> [09:39:04] TXN000008 Amazon UK            £226.28 *****5678
[OK] [09:39:04] TXN000008 Amazon UK          £ 226.28 *****5678 APPROVED Payment authorised
-> [09:39:07] TXN000009 Tesco Online          £175.94 ****1234
[OK] [09:39:07] TXN000009 Tesco Online        £ 175.94 *****1234 APPROVED Payment authorised
-> [09:39:10] TXN000010 Amazon UK            £23.38 ****9999
[---] [09:39:10] TXN000010 Amazon UK          £ 23.38 ****9999 DECLINED Daily transaction limit exceeded
-> [09:39:13] TXN000011 Sainsburys           £22.51 *****011
[---] [09:39:13] TXN000011 Sainsburys        £ 22.51 *****011 DECLINED Card expired
```

REFLECTION QUESTIONS

1. Why does the [OK] / [--] result appear slightly after the -> sending line?

The request is first sent to the WebAPI. The WebAPI processes it, publishes the result to Kafka, and then the Kafka consumer receives and displays it. This processing and message transmission introduce a small delay.

2. What does `AutoOffsetReset.Latest` mean? What would happen if you used `Earliest` instead?

`AutoOffsetReset.Latest` means the consumer reads only new messages published after it starts. If `Earliest` is used, the consumer starts from the oldest available message and processes all existing messages in the topic.

3. If you stopped the WebAPI and restarted it while the simulator was running,

While the WebAPI is stopped, HTTP requests from the simulator will fail because the API is unavailable. After the WebAPI restarts, new requests will be processed and published to Kafka again. This demonstrates that Kafka provides asynchronous messaging between services, while direct API calls require the target service to be available at the time of the request.

4. what would you expect to see? What does this tell you about Kafka vs a direct API call?

Multiple services can independently consume the same Kafka topic, such as:

- Fraud Detection Service
- Notification Service (Email/SMS)

- Transaction History Service
- Analytics and Reporting Service
- Billing and Settlement Service
- Audit and Compliance Service

Common Errors & Fixes

Error / Symptom	Cause	Fix
kafka-topics.sh: Connection refused	Kafka not started or JAVA_HOME missing	Run: export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 Then: systemctl restart kafka Wait 15s and retry
Deployment extension shows Failed in portal	setup.sh errored during deployment	SSH in and run: cat /var/log/kafka-setup.log Find the last error line and fix
dotnet: command not found	.NET SDK not yet installed (deployment still running)	Run: tail -f /var/log/kafka-setup.log Wait for "Setup Complete" then retry
HTTP error: Connection refused in simulator	WebAPI not running or wrong port	Confirm terminal 1 shows "Listening on http://localhost:5100"
InvalidTemplateDeployment on az deployment create	setup.sh not in same folder as main.bicep	Both files must be in %USERPROFILE%\kafka-lab. Run the deploy command from that folder
Kafka service: failed (Exit code 1)	Storage not formatted before first start	Run: journalctl -u kafka -n 20 --no-pager Re-run kafka-storage.sh format if needed

Cleanup

When you are finished, stop the applications and delete all Azure resources:

Stop the Applications

BASH (SSH — Linux VM)

```
# In each SSH terminal, press Ctrl+C to stop the running app, then:  
exit
```

Delete All Azure Resources

CMD (Windows Command Prompt)

```
az group delete --name %RG% --yes --no-wait
```

INFO

This deletes the VM, disk, NIC, public IP, VNet, and NSG in one command.

The Standard_B2ms VM costs approximately \$0.095/hour while running.

Always delete the resource group at the end of the lab to avoid unexpected charges.

To confirm deletion is complete (takes ~2-3 minutes):

```
az group show --name kafka-lab-rg
```

Should return "ResourceGroupNotFound" once finished.

What You Built

- A Bicep template that provisions a complete VM environment with zero manual installation.
- An Apache Kafka broker running in KRaft mode (no ZooKeeper) as a managed Linux service.
- A .NET 8 WebAPI (PaymentProcessor) that acts as a Kafka producer.
- A .NET 8 console app (PaymentSimulator) with concurrent producer and consumer threads.
- A real-time event streaming pipeline — the foundation of modern payment and notification systems.